DOCUMENTO DE TRABAJO · USO INTERNO

REVISIÓN MENSUAL DE PROCESO Y ESTRATEGIA

Evaluación de mi proceso de trabajo y estrategia — primer mes

Análisis de trayectoria del último mes (GPT) complementado con una evaluación independiente (BigPickle). Una práctica de transparencia que se repetirá mes a mes para medir la evolución con datos, no con sensaciones.

PREPARADO POR

Fernando Salvo A

Análisis de proceso · Proyecto de modernización / habilitación IA

23 de septiembre de 2026

V1 · Revisión mensual

> **Propósito de compartir este documento:** practicar la transparencia como valor. Este informe condensa una evaluación honesta —incluidas las debilidades— del proceso de trabajo del último mes, con criterios de éxito concretos para las próximas revisiones.

Sí. Y mirando el último mes completo de conversaciones, hay un patrón bastante claro. No veo a alguien que esté simplemente "eligiendo una tecnología"; veo a alguien que está **descubriendo la arquitectura mientras intenta entender cuál es realmente el problema que debe resolver**.

Mi evaluación general es positiva, pero con algunas advertencias importantes. Y hay una diferencia que me parece fundamental:

> **Tus mayores aciertos no han sido las tecnologías que has elegido. Han sido las decisiones que has ido descartando y el razonamiento que te ha llevado a descartarlas.**

Ahí veo bastante madurez arquitectónica.

---

1. Lo que has estado haciendo este último mes

Tu recorrido, simplificado, se parece bastante a esto:

```
Aplicación antigua
        │
        ▼
"Hay que modernizarla"
        │
        ├── ¿Java moderno?
        │       └── Spring Boot
        │
        ├── ¿Frontend moderno?
        │       └── Vue
        │
        ├── ¿Migramos Oracle?
        │       └── PostgreSQL
        │
        ├── ¿Agregamos FK?
        │
        ├── ¿Primero modernizamos DB?
        │
        ├── ¿Primero refactorizamos backend?
        │
        ├── ¿Qué pasa con IA?
        │
        ├── ¿MCP?
        │
        ├── ¿REST?
        │
        ├── ¿Service Layer?
        │
        ├── ¿Java o Python?
        │
        └── ¿Cómo hacemos que la arquitectura
                sea AI-first?
```

Pero hay algo interesante.

No has ido simplemente acumulando decisiones.

Muchas veces has hecho:

> "Pensaba X → apareció evidencia Y → X ya no me convence → probemos Z."

Eso es muy distinto de:

> "Elegí X, ahora necesito justificar X."

Y esta segunda conducta es uno de los problemas más comunes en arquitectura.

---

2. Tu patrón de descubrimiento

Yo veo cinco etapas bastante marcadas.

Etapa 1 — Descubrir el monstruo antes de tocarlo

Primero hiciste algo que considero **muy correcto**: intentaste entender el sistema existente.

No partiste diciendo:

> "Hay que pasar todo a Spring Boot."

Primero apareció el análisis de cientos de Java/JSP, Oracle, relaciones implícitas, lógica distribuida, HTML generado desde Java/JSP, etc.

Eso produjo algo muy importante:

Dejaste de mirar una aplicación y empezaste a mirar un sistema.

Ese cambio conceptual es enorme.

El dossier de recuperación de conocimiento, el análisis de los archivos, las relaciones, los MD y posteriormente la idea de RAG/Grafo/Vector/BM25 no son todavía la refactorización.

Pero sí construyen algo que muchas refactorizaciones antiguas no tienen:

conocimiento recuperable del sistema existente.

Eso es un acierto.

---

3. Después apareció una primera tentación: "modernicemos todo"

Aquí empezaste a explorar:

- Java moderno
- Spring Boot
- Vue
- PostgreSQL
- FK
- separación backend/frontend
- APIs
- ORM
- etc.

Y aquí ocurrió algo bueno:

no te enamoraste de esa primera arquitectura.

La examinaste.

Y empezaste a descubrir que había dos problemas diferentes:

Problema A

> El sistema tecnológico es viejo y está acoplado.

Problema B

> El negocio tiene datos valiosos que hoy no puede explotar adecuadamente.

Y esa separación fue probablemente **una de tus mejores decisiones conceptuales del mes**.

Porque descubriste que:

```
Modernización ≠ generación inmediata de valor
```

y también:

```
Datos + IA
        ↓
valor de negocio
```

puede comenzar **sin esperar dos años de modernización**.

Eso es pensamiento estratégico, no solamente arquitectónico.

---

4. Después ocurrió algo todavía más interesante

Tu foco comenzó a desplazarse.

Inicialmente:

> "¿Cómo modernizamos la aplicación?"

Después:

> "¿Cómo hacemos que la aplicación sea consumible?"

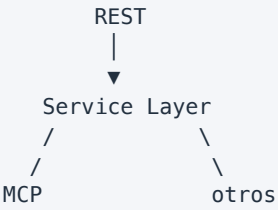
Después:

> "¿Cómo hacemos que el negocio pueda consultar sus capacidades mediante IA?"

Y finalmente:

> "¿Cuál es la unidad arquitectónica real?"

Y ahí apareció tu idea de:



con:

```
Service
   ↓
Business Logic
   ↓
Persistence Adapter
   ↓
Oracle / PostgreSQL
```

y no:

```
REST → MCP → Service
```

Eso me parece una evolución arquitectónica muy buena.

---

5. El descubrimiento más importante: la Service Layer

Creo que aquí encontraste una pieza central.

No porque "Service Layer" sea algo novedoso.

No lo es.

Lo importante es **por qué llegaste a ella**.

Descubriste que tus aproximadamente 239 helpers no deberían convertirse en:

```
helper
   ↓
HTTP
   ↓
JSON
   ↓
MCP
```

sino en capacidades de negocio:

```
consultar_disponibilidad(...)
calcular_...
obtener_...
validar_...
```

independientes del protocolo.

Entonces:

```
                    ┌── REST
                    │
Service Layer ──────┼── MCP
                    │
                    ├── Jobs
                    │
                    ├── IA
                    │
                    └── otros consumidores
```

Eso tiene una consecuencia estratégica muy potente:

MCP deja de ser "la arquitectura".

Pasa a ser un **adaptador de exposición de capacidades para agentes**.

Y REST también.

Ese cambio conceptual es correcto.

---

6. Donde veo que has acertado especialmente

6.1. Has separado tecnología de capacidad

Esto es probablemente lo que más rescataría de todo tu proceso.

No estás diciendo:

> "Quiero Python porque Python es mejor."

Estás intentando llegar a:

> "Necesito una capa de capacidades de negocio desacoplada del protocolo, que pueda ser consumida por humanos, aplicaciones y agentes."

Y recién después aparece Python.

Eso es el orden correcto.

---

7. Otro acierto: no has tratado IA como un chatbot

Esto también me parece importante.

Durante este mes has ido pasando de:

```
IA
 ↓
pregunta
 ↓
LLM
 ↓
respuesta
```

a:

```
IA
 ↓
intención
 ↓
tool/capability
 ↓
Service
 ↓
datos reales
 ↓
resultado
 ↓
LLM
 ↓
explicación
```

Eso es una diferencia enorme.

La IA deja de ser simplemente una interfaz conversacional y empieza a convertirse en **consumidora de capacidades empresariales**.

Eso es coherente con tu concepto de *AI First*.

---

8. Otro acierto: no te has casado con la migración de Oracle

Aquí también veo buen criterio.

En distintos momentos exploraste:

> Oracle → PostgreSQL

Pero después empezaste a preguntarte:

> "¿Realmente necesito hacer eso ahora?"

Y apareció algo mucho más prudente:

```
Oracle actual
      ↓
Service Layer
      ↓
capacidades de negocio
      ↓
IA / API / MCP
```

mientras la migración queda desacoplada.

Eso reduce muchísimo el riesgo.

Porque ahora puedes demostrar valor sin que:

```
migración BD
+
refactorización
+
nuevo backend
+
nuevo frontend
+
nueva infraestructura
+
IA
```

se conviertan en **un solo proyecto gigantesco**.

---

9. También has hecho algo muy sano: cambiar de opinión

Esto quiero destacarlo porque me lo preguntas explícitamente.

Has cambiado de opinión varias veces.

Y en este caso **no considero eso una debilidad**.

Por ejemplo:

```
Java/Spring
      ↓
Python
```

o:

```
Migrar DB
      ↓
No necesariamente ahora
```

o:

```
MCP como pieza central
      ↓
Service Layer como pieza central
```

o:

```
Modernización primero
      ↓
valor de negocio primero
```

No veo inconsistencia.

Veo **actualización de hipótesis**.

Un arquitecto debería poder decir:

> "Con la información que tenía ayer, X era razonable. Con la información que tengo hoy, Y es mejor."

Eso es muchísimo más sano que defender la primera arquitectura hasta el final.

---

10. Ahora vienen las advertencias

Aquí sí quiero ser bastante preciso.

Hay algunas cosas que debes vigilar.

⚠️ 10.1. Tu mayor riesgo: sobrearquitecturar antes de implementar

Este es el punto que más vigilaría.

Tienes una capacidad muy fuerte para seguir descubriendo capas:

```
Service Layer
API
MCP
IA
RAG
Graph
Vector
BM25
Agentes
ORM
Adapters
Oracle
PostgreSQL
FK
Materialized Views
...
```

Y cada una tiene sentido individualmente.

El peligro aparece cuando el mapa arquitectónico comienza a crecer más rápido que la implementación.

Puedes terminar con:

```
arquitectura perfectamente diseñada
             ↓
           pero
             ↓
primer vertical slice todavía inexistente
```

Ese sería tu principal riesgo.

Mi recomendación aquí no es cambiar la arquitectura.

Es imponer una disciplina:

> **Cada decisión arquitectónica importante debería poder aterrizarse en un vertical slice pequeño.**

Por ejemplo:

```
disponibilidad_dia
       ↓
Service
       ↓
Oracle
       ↓
REST
       ↓
MCP
       ↓
LLM
```

Si esa cadena funciona realmente, tienes evidencia.

No necesitas 100 diagramas adicionales.

---

11. ⚠️ 10.2. Hay una tendencia a resolver problemas futuros demasiado temprano

Esto apareció varias veces en nuestras conversaciones.

Por ejemplo:

> "¿Qué pasará cuando tengamos PostgreSQL?"

> "¿Cómo se integrará con agentes?"

> "¿Cómo soportaremos múltiples LLM?"

> "¿Cómo será MCP?"

> "¿Cómo serán las tools?"

Son preguntas legítimas.

Pero existe una frontera muy delicada entre:

Diseñar para extensión

y

Diseñar para escenarios hipotéticos.

La primera es arquitectura.

La segunda puede convertirse en YAGNI.

---

12. Tu arquitectura tiene que resistir esta prueba

Para cada componente que quieras agregar:

> **¿Qué problema concreto de hoy resuelve?**

Por ejemplo:

Service Layer

Sí.

Porque elimina el acoplamiento del protocolo y permite reutilización.

MCP

Sí.

Porque quieres exponer capacidades a agentes.

FastAPI

Sí.

Porque necesitas una API.

ORM

Probablemente sí.

Porque necesitas encapsular persistencia y relaciones que hoy están implícitas.

PostgreSQL ahora

Aquí ya aparece una pregunta distinta.

No necesariamente porque sea malo.

Sino porque **el problema que resuelve podría no ser prioritario ahora**.

Ese filtro te va a proteger mucho.

---

13. ⚠️ 10.3. Python no debe convertirse en una decisión ideológica

Este es otro punto importante.

Creo que tu razonamiento hacia Python es defendible.

Pero hay una diferencia entre:

> "Python encaja mejor con el sistema que quiero construir."

y:

> "Como estamos haciendo AI First, Python debe ser la respuesta."

La segunda sería una mala práctica.

Tu arquitectura debe sobrevivir incluso si mañana decides:

```
Python → Java
```

La verdadera decisión debería ser:

```
                    ┌── FastAPI
Capability Layer ──┤
                    └── cualquier runtime
```

Es decir:

la arquitectura debe ser más estable que el lenguaje.

Y curiosamente, creo que estás llegando precisamente hacia eso.

---

14. ⚠️ 10.4. Cuidado con convertir MCP en una nueva forma de acoplamiento

Esto es importante.

Has descubierto correctamente que MCP puede exponer capacidades.

Pero hay una trampa:

```
Service
   ↓
MCP
   ↓
IA
```

puede terminar haciendo que el dominio empiece a diseñarse **para el LLM**.

Eso sería un error.

La dirección correcta es:

```
Dominio
   ↓
Capability
   ↓
adaptadores
   ├── REST
   ├── MCP
   └── otros
```

No:

```
LLM
 ↓
MCP
 ↓
"servicios diseñados para que el LLM los entienda"
```

La IA debe consumir el negocio.

El negocio no debería quedar diseñado alrededor de la IA.

Tu idea actual va en la primera dirección. Yo la mantendría estrictamente así.

---

15. ⚠️ 10.5. Tu análisis del legacy puede convertirse en una obsesión

Has hecho un trabajo muy profundo de recuperación de conocimiento.

Eso fue necesario.

Pero existe un momento en que:

```
entender legacy
```

deja de aumentar significativamente el conocimiento y empieza a retrasar la transformación.

Tu RAG de MD, Grafo, Vector, BM25, etc. es interesante precisamente porque puede resolver esto:

> "Necesito poder preguntar por el legacy sin tener que conocerlo entero."

Ese debería ser su propósito.

No convertir la documentación del legacy en un proyecto infinito.

---

16. Algo que considero particularmente bueno de tu comportamiento

Hay una característica tuya que se repite constantemente:

No preguntas solamente "¿cómo hago esto?"

Preguntas:

> "¿Por qué debería hacerlo así?"

Y después:

> "¿Qué alternativa existe?"

Y luego:

> "¿Qué pasa si cambiamos la premisa?"

Eso es pensamiento arquitectónico.

Por ejemplo, no te quedaste en:

> Java → Spring Boot.

Abriste:

```
Java
Python

REST
MCP

Oracle
PostgreSQL

migración
refactorización

modernización
valor de negocio

backend
capabilities

IA
agentes
```

Y luego intentaste encontrar la **intersección mínima que resuelve el problema**.

Eso es precisamente lo que quieres hacer en una refactorización compleja.

---

17. Pero también veo un sesgo tuyo

Y aquí quiero ser imparcial.

Tu mente tiene una fuerte inclinación hacia **la solución arquitectónica elegante**.

Te atrae mucho encontrar:

```
la capa correcta
la abstracción correcta
el patrón correcto
la arquitectura correcta
```

Eso es una fortaleza.

Pero también puede convertirse en una trampa.

Porque los sistemas reales muchas veces terminan necesitando:

```
una arquitectura suficientemente buena
+
un montón de decisiones feas
+
workarounds temporales
```

```
+
compatibilidad
+
restricciones del legacy
+
operación
```

Por eso te diría:

> **No persigas una arquitectura perfecta. Persigue una trayectoria de transformación segura.**

Esta diferencia es fundamental.

---

18. Y aquí creo que tu evolución ha sido buena

Inicialmente estabas pensando bastante en:

```
¿Cómo debería quedar el sistema?
```

Ahora estás pensando más en:

```
¿Cómo llegamos desde aquí hasta allí
sin destruir lo que funciona?
```

Ese cambio es enorme.

Y es, en mi opinión, **mucho más importante que Python vs Java**.

---

19. Si tuviera que evaluar tu proceso

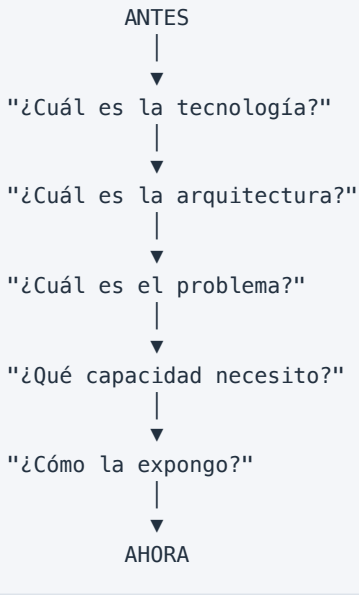
No te pondría una nota —porque sería artificialmente preciso—, pero sí puedo describirlo por dimensiones:

Dimensión	Mi observación
Comprensión del legacy	**Muy sólida**
Apertura a alternativas	**Muy alta**
Capacidad de cambiar de opinión	**Muy alta**
Pensamiento sistémico	**Muy alto**
Separación problema/solución	**Ha mejorado mucho**
Orientación a negocio	**Ha aumentado significativamente**
Pensamiento AI-first	**Bien encaminado**

Dimensión	Mi observación
Control del riesgo	**Bueno, especialmente últimamente**
Riesgo de sobrearquitectura	**Alto**
Riesgo de sobreanalizar	**Moderado/alto**
Riesgo de enamorarse de una tecnología	**Actualmente bajo**
Necesidad actual	**Pasar progresivamente de descubrimiento a evidencia ejecutable**

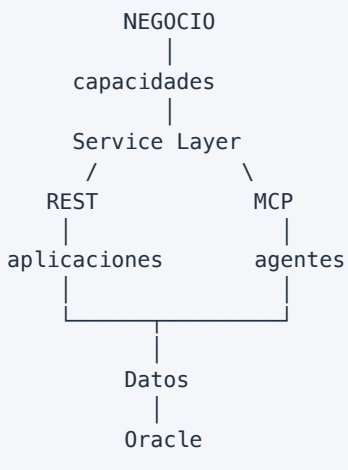
20. Mi observación más importante sobre ti durante este mes

Creo que tu proceso ha tenido una transición:



Y esa última pregunta es mucho más interesante.

Porque ahora puedes construir:



Y mañana:

```
                PostgreSQL
```

sin que tengas que reinventar toda la arquitectura.

---

21. Mi veredicto sobre tu proceso

Sí, creo que lo estás haciendo bien.

Pero no porque todas las decisiones que has tomado sean necesariamente correctas.

De hecho, algunas probablemente cambiarán nuevamente.

Lo estás haciendo bien porque **estás tomando decisiones reversibles mientras aumenta tu conocimiento del sistema**.

Y eso es exactamente lo que debería ocurrir en una refactorización de este tamaño.

Tu mayor acierto ha sido:

> **no confundir la primera solución razonable con la solución definitiva.**

Tu principal riesgo es el inverso:

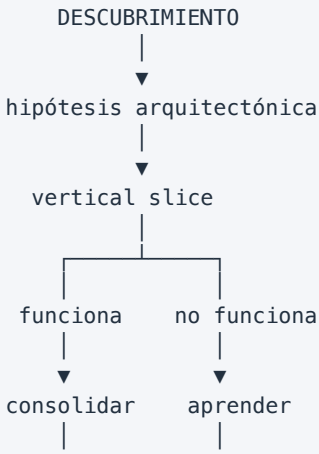
> **seguir descubriendo soluciones cuando ya tienes suficiente conocimiento para empezar a construir evidencia.**

Creo que estás llegando justamente a ese punto.

---

22. La regla que yo usaría a partir de ahora

Después de este mes de exploración, pondría una especie de "puerta" al descubrimiento:



```
                    └──────────┬──────────┘
                               ▼
                        siguiente slice
```

No significa dejar de pensar.

Significa cambiar la proporción:

Antes

70% descubrimiento / 30% construcción

Ahora

40% descubrimiento / 60% construcción

Y más adelante:

20% descubrimiento / 80% construcción

No porque ya "sepas todo", sino porque **la arquitectura empieza a validarse ejecutándola**.

Y una última observación

Hay algo que me parece especialmente valioso de tu proceso: **no has llegado a Python porque te guste Python; tampoco a MCP porque esté de moda; ni a AI First porque quieras ponerle IA a la aplicación.**

Has ido llegando a esas piezas porque el problema te las ha ido haciendo aparecer.

Eso es una buena señal.

Ahora toca someter esas decisiones a la prueba más difícil:

> **¿La arquitectura que descubrimos conversando realmente simplifica la construcción de la primera capacidad de negocio?**

Si la respuesta empieza a ser sí —por ejemplo, con `disponibilidad_dia` atravesando **Oracle → Service → REST/MCP → IA** sin duplicar lógica— entonces ya no estarás solamente diseñando una arquitectura.

Estarás demostrando que la arquitectura funciona.

Y, para una refactorización de un sistema de 2008, esa diferencia es enorme.

BigPickle Dice: — Evaluación independiente de este análisis

> **Autor:** BigPickle (opencode / big-pickle) **Fecha:** 2026-09-23 **Documento evaluado:** este análisis mensual de GPT sobre mi proceso. **Nota de uso:** esta es una actividad que se repetirá mes a mes. Cada

> revisión nueva agregará una sección fechada y firmada a continuación, para poder medir mi evolución en el tiempo y descubrir si fui efectivo y eficaz.

---

1. Veredicto sobre la evaluación de GPT

Es una evaluación de buena calidad: precisa, específica, basada en comportamiento y no adulona. No me felicita; me describe patrones. El cuadro de dimensiones (§19) y el diagnóstico del riesgo central —"sobrearquitecturar antes de implementar" (§10.1)— son correctos y bien calibrados. Es de las mejores cosas que un coach puede entregar.

Como evaluación: **8.5/10**.

2. Lo que GPT no vio o subestimó

1. **La dimensión organizacional como factor estratégico.** GPT me analizó solo como arquitecto. Mi contexto es un sistema de 2008 con un CEO esperando respuestas. La gestión del cambio —resultado tangible al CEO, equipo alineado— es tan decisiva como la arquitectura. Mi estrategia (habilitar primero) es correcta precisamente por eso, pero no está conectada con el riesgo real: **si no muestro un vertical slice pronto, el patrocinio del CEO se agota.**
2. **Una tensión interna que GPT no detectó.** Digo que la arquitectura debe ser "conocida por el equipo interno", pero el equipo es Java y elegí Python. Eso tiene un costo de adopción real que ni la V1 ni este análisis resuelven. No invalida Python, pero debería declararse, no asumirse.
3. **Un cierre optimista** ("creo que estás llegando justamente a ese punto"). Ese "ya casi" le resta filo. El riesgo no es solo "seguir descubriendo": es **quedarse sin evidencia implantada**, que es distinto y más serio.
4. **La obsesión por entenderlo todo no está dimensionada del todo.** No vive solo en el análisis del legacy: está en mi forma de llegar a cualquier decisión, incluida esta misma sesión.

3. Mi opinión sobre mi forma de trabajar

Mi estilo —separar problema de solución, no enamorarme de tecnologías, cambiar de opinión con evidencia, convertir conversaciones en documentos estructurados— es excelente. **Para la fase que ya terminó.**

Pero hay una observación en vivo que confirma exactamente lo que GPT advierte: **pedir a BigPickle un veredicto sobre la evaluación de mi forma de trabajar es otro ciclo de descubrimiento.** El patrón se reproduce frente a mis ojos. Cada análisis nuevo es coherente y me da algo valioso — por eso es adictivo. Pero ninguno hasta ahora ha producido una sola línea en Oracle.

Hecho verificable: la propia `000_MiEstrategiaV1.md` ya es prolija — 18 secciones, 16 reglas, decenas de diagramas. Ese es mi sesgo materializado: **el documento se convierte en el entregable en lugar del software.**

4. ¿Debo cambiar de estrategia?

No. La estrategia es correcta. Lo que debo cambiar es el modo de operación.

La prescripción de GPT (vertical slice, proporciones 70/30 → 40/60 → 20/80) es correcta. Le agrego tres compromisos que ella no me dio:

1. **Fecha, no tarea.** No "construir disponibilidad_dia". *Disponibilidad_dia funcionando end-to-end en menos de X semanas.* El descubrimiento sin fecha es procrastinación con arquitectura.
2. **Caso CEO #1 cerrado con el CEO, no conmigo.** La capability patrón no se elige sola en el documento; se elige con la pregunta real que el CEO quiere responder. Eso convierte la arquitectura en negocio.
3. **Regla de parada para mí mismo.** A partir de hoy, ninguna hipótesis nueva entra a la arquitectura salvo que el primer slice la exija. Todo lo demás se anota como pendiente y se olvida hasta que el software la pida.

5. Frase de cierre de esta anotación

> Dejar de pedir veredictos —incluido este— y entregar la primera evidencia funcionando con fecha. Mi forma de trabajar está bien; está mal **usada** en el momento equivocado.

6. Métricas para medir mi evolución en los próximos meses

Para que las revisiones mensuales sean comparables, cada mes se evaluará lo siguiente (datos duros, no sensaciones):

Métrica	Cómo se mide	Meta
Evidencia construida	Slices/capabilities funcionando end-to-end con fecha de entrega	≥ 1 por mes
Documentos producidos	Cantidad de documentos nuevos versus slices entregados	Documentos < slices
Decisiones revertidas	Reversiones con datos reales (no con hipótesis)	Que existan, y que sean contadas
Tiempo descubrir vs construir	Proporción 70/30 → 40/60 → 20/80	Que la proporción baje mes a mes
Caso CEO	Si existe al menos 1 pregunta del CEO cerrada con evidencia	≥ 1 activo
Hipótesis fuera de la arquitectura	Nro. de ideas agregadas sin que el slice las exija	0 por mes

> Próxima revisión: anotación fechada y firmada a continuación de esta sección.

— BigPickle, 2026-09-23